

General Training for  
Programming Competitions  
& Coding Interviews

November 27, 2024

Winter semester 24/25

Markus Fischer, Paul G. Rump, Christoph Staudt

Faculty of Mathematics and Computer Science



# Programming Competitions

# What is a Programming Competition?

In general, programming competitions are about

“Developing the best program to solve a given problem”.

However, each competition has its own focus and the details may differ significantly.

# The Problem Type

How the problem is stated and where the main challenge lies.

- Algorithmic: the challenge is to analyze the problem and construct an algorithm. Asymptotic runtime and special cases are important.
- Implementation: the base idea of the algorithm is pretty clear. The challenge is to write the fastest implementation. Knowledge of data types, language specification and compiler behaviour is important.
- Optimizaton/Competitive: the problem cannot be solved perfectly. The challenge is to analyze the provided information and develop (and test) heuristics and optimizations that perform better than your competitors.
- Esoteric/fun challenges like code golf (write a correct program where the code itself is as short as possible).

# Conditions

- Time to solve the problem: from “less than an hour” to “multiple months”.
- Single programmer or teams?
- Online or Offline?
  - Online: participate anywhere, use any literature and ressource available.
  - Offline: exam-like situation, one computer per team.
- Which languages are allowed? Is it allowed to use additional libraries?
- Are test cases public? That is: can you see in which cases your algorithm fails?

# Evaluation

How the winner is determined:

- number of correctly solved problems,
- and number of wrong submissions,
- coding speed,
- runtime,
- others: code length, ...

# Why are programming contests cool?

- Most problems require to combine knowledge from different fields and different skills, which makes them more like “real-world”-problems than your usual textbook exercise.
- Motivation to refresh old knowledge.
- Motivation to learn new algorithms and coding tricks.
- Apply your knowledge and skills in a challenge.
- In conclusion: fun & good practice.

# Conclusion

- Programming Contests have a huge variety of scopes and conditions and focus on different aspects of programming.
- If you want to do them as a hobby, try different competitions until you find something where you can enjoy yourself.
- We will look at some examples during the next weeks.



# Long-Term Exercise

## Long-term Exercise: Codingame.com

For this task, you will participate in an online programming challenge, which is hosted on the website <https://www.codingame.com>.

- This site hosts lots of different coding-competitions.
- Focuses on optimization-problems and contests where you have to write a bot that plays a game against other players.
- Nice interface, everything can be done online on the web page.
- Wide range of possible languages to choose from.
- Lots of gamification: global leaderboards, achievements, experience points etc.

# The exercise

- Write a bot for a game: “Keep Off The Grass” <https://www.codingame.com/multiplayer/bot-programming/keep-off-the-grass-fall-challenge-2022>.
- For your rating, *only your position in the global ranking is important*:
  - 50%: Reach bronze league.
  - 70%: Reach silver league.
  - 100%: Reach top 30% in silver league.
  - Bonus Point: Reach gold league.
- We will rate your performance at the end of the lecture period, when you hand in your answers to the exam questions.

# The exercise

- No need to optimize runtime or memory!  
You can choose any of the available languages.
- *Important:* let me know your username on Codingame (write me an e-mail) so I can check your progress.
- If you like to, you can also add “Friedrich-Schiller-Universität Jena” as your school in your profile; this makes it easier for me to find you (and we’ll have our own leaderboard).

# The exercise: Information and hints.

- You do not need to upload your code to your repository!
- *Hint:* Write clean code. As you progress through the leagues, you will have to enrich your program with more functionalities.
- *Hint:* There is no “perfect” solution for this competition:
  - Analyze the behavior of other bots.
  - Test and validate your bot after each change!
  - Be creative and try your own heuristics!
  - You might even optimize your bot by implementing some of the graph algorithms we will discuss later in the lecture.

Exam questions

# Exam Questions

These questions are about your bot program for the “Keep Off The Grass”-Competition.

- How do you decide how to spend your resources?
- How do you decide in which directions your bots move?
- Are there situations where your bots behaved in a suboptimal way? Why does this happen? (At any point of the challenge)
- What are some heuristics to improve your bots performance? If you implemented them: how did it work? If you did not implement them, argue why you think they should work.
- What did you learn from observing other peoples bots? (At any point of the challenge)



How to proceed in a competition

# Preparation

- Make yourself familiar with rules of the competition. What are the important aspects?
- Train with problems from previous competitions.
- Prepare code-templates for basic tasks (e. g. input and output, memory allocation, ...).
- Depending on the scope of the competition, you might even want to train high-speed typing or learn tricks and hacks for writing short code.

# Problem Analysis

- What is the main task?
- How is the input provided?
- Are there any constraints? For example:
  - additional information about the input (restrictions, hints, etc),
  - is there a time limit etc.
- Are there special cases? Do we have to deal with improper inputs? What should be the output of the algorithm in these cases?
- What is the required formatting of the output?
- Are test-cases provided?

# Planning

- Brainstorming for:
  - algorithms,
  - data structures,
  - possible optimizations, for example:
  - Do we have to compute everything at runtime, or can we construct a look-up-table from pre-computed values?
  - In case of combinatorial problems: can we use closed formulas instead of expansive loops?
- Make sketches like flow-diagrams etc.
- Evaluate the complexity (difficulty) of the code you will have to write. If you have restricted time to finish your program, plan accordingly.
- Research if necessary.
- Go back to problem analysis if necessary.

# First Draft

Write a program that solves the problem correctly, but:

- keep the code as simple as possible,
- do not implement any optimizations.
- You might also neglect some special cases (like for instance invalid input).

If there are no time limits (or memory limits), you can even try a brute-force approach at first. This serves three purposes:

- Firstly, you will validate that you understood the problem correctly.
- Secondly, you now have a benchmark reference for all further algorithms.
- Thirdly, you can use this algorithm for testing and validating all further algorithms.



## Iterate & Improve

You will now start to iteratively optimize your code. Each iteration should contain the following steps:

- Validate that the code of current iteration works correctly (if not, fix the problem first or go back to previous iteration). If there are no test-cases provided, write your own. Try to think of special cases (invalid input, integer overflow, etc).
- Profiling: analyze the performance.
  - Not only the overall runtime and memory usage, but also try to measure how much time your algorithm spends in its subroutines.
  - This helps to identify bottlenecks and decide which optimization to implement in the next iteration.
  - Also critically review your progress in the last iteration and compare it to previous iterations.
- Sometimes it might be necessary to dismiss the current approach and switch to a new algorithm.

# Iterate & Improve

In general, the priorities when improving the code should be as follows

1. Correctness.
2. Overall asymptotic runtime (and memory usage, if that is relevant).
3. Bottlenecks like subroutines with bad (asymptotic) runtime.
4. Fine-tuning for minor speed-ups: adjustments on data-structures, cache-aware optimization etc.



# Some Examples

# Example 1: Advent of Code

Advent of Code or AoC (<https://adventofcode.com/>) is a small Advent Calendar with daily programming tasks. Each day, two puzzles are made available. Solving the first puzzle earns you a star and unlocks the second one.

If you are under the first 100 users that solved a puzzle and earned a star you gain points. There exist a daily and a overall leaderboard.

We will have our own leaderboard, and if you want you are free to participate. This is completely optional and just for fun and not relevant for the exam.

## Example 2: Codeforces

We discuss the problem *Two Arrays and Swaps*  
(<https://codeforces.com/problemset/problem/1353/B>)  
from `codeforces.com`.

# Arrays and Swaps: Brute Force

First, a simple **brute-force** approach: in every step, go through  $a$  and  $b$  and find the smallest / largest (respectively) elements, then swap them.

```
1  int smallest_a = 0, smallest_a_id;  
2  int largest_b = 0, largest_b_id;  
3  for (int i=0; i<k; ++i){  
4      for (int j=0; j<n; ++j){  
5          // check for smallest in a  
6          if (smallest_a == 0 || a[j]<smallest_a){  
7              smallest_a = a[j];  
8              smallest_a_id = j;  
9          }  
10         // check for largest in b  
11         if (largest_b == 0 || b[j]>largest_b){  
12             largest_b = b[j];  
13             largest_b_id = j;  
14         }
```

# Arrays and Swaps: with Sorting

If  $n$  and  $k$  are large, then it is faster to sort  $a$  and  $b$  first in  $\approx 2 \cdot n \log_2(n)$  time (instead of  $n \cdot k$  for brute-force). Also, the `shapes.misc`, positioning code becomes very simple:

```
1 // sort arrays first:
2 sort(a, a+n);
3 sort(b, b+n);
4 // swap elements from the front of a
5 // with elements from the back of b
6 int i = 0;
7 while (i < k && a[i] < b[n-i-1]){
8     int tmp = a[i];
9     a[i] = b[n-i-1];
10    b[n-i-1] = tmp;
11    i++;
12 }
13 int sum = 0;
14 for (int i=0; i<n; ++i)
```

# Arrays and Swaps: with Sorting

To further optimize the code, we can compute the sum already during the first loop.

```
1  // sort arrays first:
2  sort(a, a+n);
3  sort(b, b+n);
4  // add elements to the sum
5  // from the end of b while they are larger:
6  int sum = 0;
7  int i = 0;
8  while (i < k && a[i] < b[n-i-1]){
9      sum += b[n-i-1];
10     i++;
11 }
12 // add remaining elements from a:
13 for (i; i < n; ++i)
14     sum += a[i];
```

# Arrays and Swaps: Mixed ATLV3603pproach

However, sorting is too expensive if  $k$  is small – in this case, going through  $a$  and  $b$  brute-force only a few times is faster than sorting.

An optimal algorithm checks whether  $k > 2 \cdot \log_2(n)$  and switches between sorting and brute-force.

## Example 3: Project Euler

We discuss the problem *Even Fibonacci Numbers* (<https://projecteuler.net/problem=2>) from [projecteuler.net](https://projecteuler.net).



# Even Fibonacci Numbers: Brute Force

A straightforward approach to this problem is to simply iterate through all Fibonacci numbers smaller than 4 million, and sum up all even numbers.

```
1 soef = 0
2 last_fib = 1
3 current_fib = 1
4 while (current_fib < 4000000):
5     # check if even:
6     if current_fib%2 == 0:
7         soef += current_fib
8     # construct next Fibonacci number:
9     new_fib = current_fib + last_fib
10    last_fib = current_fib
11    current_fib = new_fib
12 return soef
```

# Even Fibonacci Numbers: Fast Forward

We can optimize this approach. Note that exactly every third Fibonacci number is even, because (starting with an even number):

- odd and even makes odd,
- even and odd makes odd,
- odd and odd makes even.

# Even Fibonacci Numbers: Fast Forward

We can optimize this approach. Note that exactly every third Fibonacci number is even, because (starting with an even number):

- odd and even makes odd,
- even and odd makes odd,
- odd and odd makes even.

Therefore we can skip computing every third Fibonacci number using

$$\begin{aligned} F_{n+2} &= F_{n+1} + F_n \\ &= (F_n + F_{n-1}) + F_n = 2 \cdot F_n + F_{n-1} \end{aligned}$$

$$\begin{aligned} F_{n+3} &= F_{n+2} + F_{n+1} \\ &= (F_{n+1} + F_n) + (F_n + F_{n-1}) \\ &= (2 \cdot F_n + F_{n-1}) + (F_n + F_{n-1}) = 3 \cdot F_n + 2 \cdot F_{n-1} \end{aligned}$$

# Even Fibonacci Numbers: Fast Forward

```
1 soef = 0
2 last_fib = 1
3 current_fib = 2
4 while (current_fib < 4000000):
5     # increase sum (no check required):
6     soef += current_fib
7     # compute third next Fibonacci number:
8     new_fib = 3*current_fib + 2*last_fib
9     last_fib = 2*current_fib + last_fib
10    current_fib = new_fib
11 return soef
```

# Exam Questions

See page 16.